Classificação de Sentimento Binário com KNIME: Construção e Análise de um Pipeline de Processamento de Texto

Luiz Filipe Brandi
Programa de Engenharia de Sistemas e Computação (COPPE/UFRJ)
Centro de Tecnologia, Cidade Universitária, Rio de Janeiro – RJ – Brazil
brandi@cos.ufrj.br

Código disponível em: `https://github.com/LuizBrandi/binary-sentiment-classification`

1 Introdução

O processamento de linguagem natural e a mineração de texto têm se tornado áreas de crescente importância, especialmente diante do grande volume de dados textuais disponíveis em diferentes domínios. Nesse contexto, tarefas como a análise de sentimentos destacam-se por sua ampla aplicação na interpretação automática de opiniões e avaliações textuais.

O KNIME destaca-se como uma plataforma *low-code* que possibilita a criação de fluxos de processamento de dados por meio de uma interface gráfica baseada em nós. Neste trabalho, investiga-se a aplicação dessa ferramenta na construção de um pipeline completo para a tarefa de classificação de sentimento binário, positivo e negativo, utilizando um conjunto de dados composto por sentenças extraídas de avaliações de produtos, filmes e restaurantes.

Ao longo do relatório, são descritas as etapas do pipeline desenvolvido que pode ser visualizado na Figura 1, incluindo o pré-processamento dos textos, a construção de representações vetoriais e a tentativa de aplicação de algoritmos de aprendizado de máquina para classificação. Além disso, são discutidas as dificuldades encontradas durante a implementação, especialmente no que se refere à manipulação e transformação dos dados dentro da plataforma, bem como suas implicações na execução do experimento.

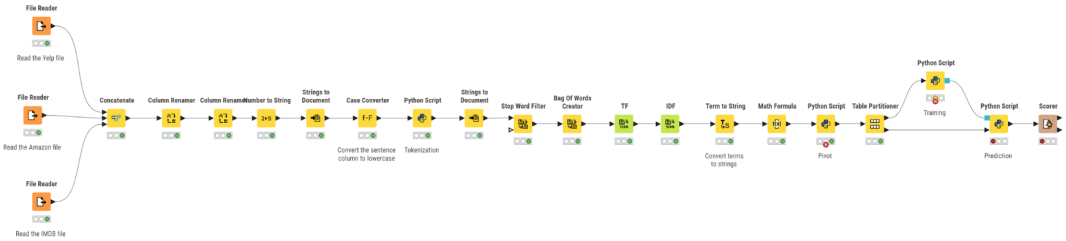

Figura 1: Pipeline do Knime para o Experimento de Classificação de Sentimento Binário.

2 Descrição da Base de Dados

A base de dados utilizada neste trabalho foi originalmente proposta por *Kotzias et al.* no artigo "From Group to Individual Labels using Deep Features" (KDD, 2015). O conjunto de dados é composto por sentenças rotuladas com sentimento positivo ou negativo, extraídas de avaliações de produtos, filmes e restaurantes.

Cada instância do dataset segue o formato *sentence* \t *score* \n, no qual *sentence* corresponde ao texto da sentença e *score* representa o rótulo associado. O valor do rótulo é binário, sendo 1 para sentimentos positivos e 0 para sentimentos negativos.

As sentenças foram coletadas de três diferentes fontes: *IMDb*, *Amazon* e *Yelp*. Para cada uma dessas fontes, o dataset contém 500 sentenças positivas e 500 negativas, totalizando 1000 sentenças por fonte. Essas sentenças foram selecionadas aleatoriamente a partir de conjuntos maiores de avaliações, com o objetivo de incluir apenas exemplos com polaridade claramente definida, evitando a presença de casos neutros.

3 Carga dos Dados

Os dados do dataset estão disponíveis no formato `.txt`. Para a etapa de carga dos dados, foi utilizado o nó *File Reader*, responsável por converter os arquivos de texto em tabelas estruturadas dentro do KNIME. Como o dataset é composto por três arquivos distintos, foram utilizados três nós *File Reader*.

Em seguida, utilizou-se o nó *Concatenate* para unificar as três tabelas em uma única estrutura de dados. Por fim, com o objetivo de melhorar a legibilidade e organização, foi empregado o nó *Column Renamer*, renomeando as colunas para "sentence" (texto) e "score" (rótulo).

4 Pré-processamento de Texto

O pré-processamento teve início com a conversão dos valores da coluna *score* para o tipo *string*, utilizando o nó *Number to String*. Essa transformação foi realizada para garantir compatibilidade com os modelos de aprendizado de máquina utilizados posteriormente.

Na sequência, foi aplicado o nó *Case Converter* para converter todos os caracteres da coluna *sentence* para minúsculas, padronizando o texto.

A etapa de tokenização apresentou dificuldades durante a execução do experimento. Inicialmente, tentou-se utilizar o nó *Spacy Tokenizer*, disponível na extensão *Text Processing* do KNIME. Esse nó requer como entrada tanto a tabela de dados quanto um modelo do *spaCy*, responsável por definir a língua e o formato da tokenização. No entanto, não foi possível obter o funcionamento esperado, pois a saída gerada continha *strings* vazias.

Como alternativa, optou-se pelo uso do nó *Python Script*, no qual foi implementado um código em Python para realizar a tokenização. Nesse código, utilizou-se a biblioteca *spaCy* com o modelo `en_core_web_sm`, realizando a segmentação dos textos em tokens e a remoção de espaços em branco.

Após a tokenização, empregou-se o nó *String to Document* para converter os textos tokenizados para o formato *Document*, que é um tipo de dado nativo do KNIME requerido por diversos nós de processamento textual. Por fim, foi realizada a remoção de *stopwords* por meio do nó *Stop Word Filter*, utilizando a lista padrão da língua inglesa disponibilizada pelo próprio KNIME.

5 Representação Vetorial

Para a representação vetorial dos textos tokenizados, foi adotada a abordagem baseada em Bag-of-Words (BoW) em conjunto com a ponderação TF-IDF (*Term Frequency–Inverse Document Frequency*). Essa escolha se justifica por se tratar de uma técnica amplamente utilizada em tarefas de classificação de texto, por sua simplicidade, eficiência computacional e capacidade de capturar a importância relativa dos termos em um conjunto de documentos.

Inicialmente, utilizou-se o nó *Bag of Words Creator*, responsável por transformar os documentos em uma representação baseada na frequência de termos. Como entrada, esse nó recebe uma coluna no formato *Document* e produz como saída uma tabela contendo os termos identificados e suas respectivas ocorrências em cada documento.

Em seguida, foram aplicados os nós *TF* e *IDF*. O nó *TF* calcula a frequência dos termos em cada documento, podendo ser normalizada em relação ao tamanho do documento, enquanto o nó *IDF* calcula o inverso da frequência de documentos em que cada termo aparece, reduzindo o peso de termos muito frequentes no corpus. Ambos os nós recebem como entrada a representação gerada pelo *Bag of Words Creator* e produzem como saída colunas adicionais contendo os valores de TF e IDF para cada termo.

Após essas etapas, utilizou-se o nó *Term to String* para converter a representação dos termos para o formato *string*, permitindo sua manipulação em etapas posteriores do fluxo. Por fim, foi empregado o nó *Math Formula* para calcular explicitamente o valor de TF-IDF por meio da fórmula `$TF rel$ * $IDF$`, combinando as informações de frequência

local e global dos termos.

A utilização da ponderação TF-IDF contribui para destacar termos mais relevantes para a discriminação entre classes, ao mesmo tempo em que reduz a influência de palavras muito comuns, tornando a representação mais adequada para algoritmos de classificação.

6 Divisão dos Dados

Após o processo de transformação dos dados em representação vetorial, a documentação do KNIME recomenda a utilização do nó *Document Vector*, responsável por converter documentos textuais em vetores numéricos em um formato compatível com os algoritmos de aprendizado de máquina disponíveis na plataforma. Esse nó recebe como entrada documentos enriquecidos com informações de termos, como TF-IDF, e produz uma tabela estruturada em formato vetorial, na qual cada linha representa um documento e cada coluna representa um atributo, isto é, um termo.

Entretanto, ao utilizar esse nó, observou-se que a estrutura dos dados foi significativamente alterada, resultando em perda de rastreabilidade entre os documentos originais e suas representações vetoriais. Essa desorganização dificultou a etapa de predição, uma vez que a correspondência entre instâncias e rótulos aparentava estar comprometida.

Diante dessa limitação, optou-se por utilizar um nó *Python Script*, denominado *Pivot*, com o objetivo de reorganizar a tabela manualmente. Nesse processo, os dados foram reestruturados de modo que cada linha representasse um documento e cada coluna um termo, preservando a correspondência com a variável alvo, *score*. Essa abordagem permitiu maior controle sobre o formato final dos dados e garantiu a consistência necessária para o treinamento dos modelos.

Em seguida, foi utilizado o nó *Table Partitioner* para dividir o conjunto de dados em subconjuntos de treinamento e teste. Esse nó recebe como entrada a tabela completa e permite especificar a proporção de divisão. Neste trabalho, adotou-se a divisão de 70% dos dados para treinamento e 30% para teste, uma escolha comum na literatura por proporcionar um equilíbrio entre aprendizado do modelo e avaliação de sua capacidade de generalização.

7 Treinamento do Modelo

Para a tarefa de classificação, foi escolhido o algoritmo *Naive Bayes*, amplamente utilizado em problemas de classificação de texto devido à sua simplicidade, eficiência computacional e bom desempenho em representações do tipo Bag-of-Words e TF-IDF. Esse algoritmo assume independência condicional entre os atributos, o que, apesar de simplificador, costuma produzir resultados satisfatórios em tarefas de análise de sentimento.

Inicialmente, tentou-se utilizar os nós nativos do KNIME, *Naive Bayes Learner* e *Naive Bayes Predictor*. O nó *Naive Bayes Learner* recebe como entrada os dados de treinamento e a coluna alvo, gerando como saída um modelo treinado. Já o nó *Naive Bayes Predictor* utiliza esse modelo para classificar novos dados, produzindo como saída as predições e, em alguns casos, probabilidades associadas.

Entretanto, esses nós dependem diretamente da saída do nó *Document Vector*, o qual apresentou problemas nas etapas anteriores. Como consequência, optou-se por substituir essa abordagem pelo uso de nós *Python Script*, nos quais foi implementado o treinamento e a predição do modelo utilizando bibliotecas externas.

Apesar dessa alternativa, não foi possível obter êxito completo no treinamento do modelo. Observou-se que, durante o processamento, a coluna de variável alvo parecia ser alterada ou interpretada de forma inconsistente, o que comprometeu a execução do algoritmo. Esse comportamento pode estar relacionado à manipulação dos dados entre diferentes formatos dentro do KNIME e à integração com scripts externos, dificultando a rastreabilidade das transformações aplicadas.

8 Discussão e Conclusão

O KNIME mostrou-se uma ferramenta interessante para a visualização e organização de pipelines de experimentos em processamento de linguagem natural e mineração de texto. Sua interface gráfica facilita a compreensão do fluxo de dados e das etapas envolvidas no processo.

Entretanto, durante a execução deste trabalho, foram encontradas diversas dificuldades que impactaram o desenvolvimento do experimento. Algumas operações relativamente simples de serem implementadas na linguagem de programação Python, tornaram-se mais complexas no KNIME, principalmente devido ao nível de abstração da ferramenta e à falta de transparência em determinados processos internos.

Além disso, inconsistências na documentação de alguns nós e dificuldades na integração entre diferentes etapas do pipeline contribuíram para os problemas enfrentados. Em particular, questões relacionadas à transformação e manutenção da estrutura dos dados dificultaram a continuidade do fluxo e a aplicação dos modelos de aprendizado de máquina.

Dessa forma, em função das limitações encontradas e do prazo estabelecido para a entrega, não foi possível concluir integralmente o experimento nem realizar as análises quantitativas previstas. Ainda assim, a experiência permitiu compreender melhor os desafios envolvidos na construção de pipelines de processamento de texto em ambientes *low-code*, bem como a importância do controle sobre as transformações de dados ao longo do processo.

9 Referências

Kotzias, D.; Denil, M.; De Freitas, N.; Smyth, P. *From Group to Individual Labels using Deep Features*. In: Proceedings of the 21th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD), 2015.